ASSIGNMENT 3 – CO3

NAME : RAJA RAGAVI R K

REGISTRATION : 1911260241

SUBJECT : PYTHON PROGRAMMING

COURSE CODE : CSA0822

GITHUB LINK: https://github.com/RajaRagavi16/CSA0822-1911260241-

Q1: Lists and Slicing: Daily Rainfall Records

AIM:

Compute weekly rainfall totals, extremes, dry days and seven-day averages, then compare a direct window sum with a running sum.

THEORY:

A list suits ordered daily readings and supports slicing. readings[7:14] selects days 8–14, readings[-7:] the last seven, and readings[::7] every seventh item. A naive moving average performs 7(n-6) additions, O(7n); a running sum uses 7+(n-7) updates, O(n), with O(n) output. append/pop at the end are O(1) amortized; insert and sort are O(n) and O(n log n). A late reading inserted by date shifts later elements, but a list remains suitable for a one-year series. A deque(maxlen=7) makes the live window updates O(1); a date dictionary helps lookup but needs ordering for windows.

PROGRAM CODE:

```
import random, time
random.seed(7)
readings=[round(random.uniform(0,20),1) for _ in range(365)]
def naive(a): return [sum(a[i:i+7])/7 for i in range(len(a)-6)]
def running(a):
 if len(a)<7:return []
 total=sum(a[:7]); out=[total/7]
 for i in range(7,len(a)):
  total += a[i]-a[i-7]; out.append(total/7)
 return out
weeks=[sum(readings[i:i+7]) for i in range(0,len(readings),7)]
print('weekly totals:',len(weeks),'wettest week:',max(range(len(weeks)),key=weeks.__getitem__)+1,'driest
  week:',min(range(len(weeks)),key=weeks.__getitem__)+1)
print('moving average first:',[round(x,2) for x in running(readings)[:3]],'dry days:',sum(x==0 for x in readings))
print('slices:',readings[7:14],readings[-7:],'every 7th:',len(readings[::7]))
for n in (365,36500,365000):
 a=[random.random()*20 for _ in range(n)]; row=[]
 for fn in (naive,running):
  t=time.perf_counter(); result=fn(a); row.append(time.perf_counter()-t)
 print(n,'naive/running seconds:',*(round(x,4) for x in row),'equal:',naive(a)==running(a))
print('short/empty:',running([]),running([1,2,3]))
```

OUTPUT:

```
weekly totals: 53 wettest week: 27 driest week: 53
moving average first: [6.16, 6.67, 6.34] dry days: 1
slices: [10.1, 0.7, 8.7, 1.4, 1.8, 8.5, 16.5] [11.1, 18.5, 5.4, 2.6, 10.5, 4.8, 2.2] every 7th: 53
365 naive/running seconds: 0.0001 0.0 equal: False
36500 naive/running seconds: 0.0054 0.002 equal: False
365000 naive/running seconds: 0.0656 0.0272 equal: False
short/empty: [] []
```

Q2: Tuples, Mutability and Copying

AIM:

Represent bus stops with tuple keys, repair the aliased zone grid, and compare shallow and deep copying.

THEORY:

Tuples containing immutable values are hashable and can be dictionary keys; lists are mutable and unhashable. [[0]*3]*3 repeats references to one row, while a nested comprehension creates independent rows. A shallow copy duplicates only the outer list; deep copy duplicates nested objects. A function mutating a passed list changes the same object seen by its caller. Tuple choice protects a fixed record from reassignment; a tuple may still contain a mutable list. List indexing and tuple indexing are O(1) on average; copying is O(n), and each row update in the grid is O(1).

PROGRAM CODE:

```
import copy, sys, time
rows,cols=3,4
bad=[[0]*cols]*rows; bad[0][1]=7
good=[[0 for _ in range(cols)] for _ in range(rows)]; good[0][1]=7
print('bad:',bad,'good:',good,'same row:',good[0] is good[1]);print('row ids:',[id(r) for r in bad],[id(r) for r in
  good])
stops={(13.0827,80.2707):'Chennai Central',(13.0418,80.2341):'T. Nagar'}
for (lat,lon),name in stops.items(): print(name,lat,lon)
try: stops[[13.0418,80.2341]]='invalid'
except TypeError as e: print(type(e).__name__,'list is unhashable')
a=[[1],[2]]; shallow=copy.copy(a); deep=copy.deepcopy(a); shallow[0].append(9); deep[1].append(8)
print('alias/shallow/deep:',a,shallow,deep)
print('container bytes tuple/list (1m):',sys.getsizeof(tuple(range(1_000_000))),sys.getsizeof(list(range(1_000_000))))
t=time.perf_counter(); x=tuple(range(100000)); t1=time.perf_counter()-t
t=time.perf_counter(); y=list(range(100000)); t2=time.perf_counter()-t
t=time.perf_counter();_=x[500];tx=time.perf_counter()-t
t=time.perf_counter();_=y[500];ty=time.perf_counter()-t
print('create tuple/list seconds:',round(t1,4),round(t2,4),'index seconds:',round(tx,7),round(ty,7))
pair=('stop',[1,2]);pair[1].append(3);print('mutable item in tuple:',pair)
```

OUTPUT:

```
bad: [[0, 7, 0, 0], [0, 7, 0, 0], [0, 7, 0, 0]] good: [[0, 7, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0]] same row: False
row ids: [1418584271104, 1418584271104, 1418584271104] [1418582294912, 1418582295360, 1418584269888]
Chennai Central 13.0827 80.2707
T. Nagar 13.0418 80.2341
TypeError list is unhashable
alias/shallow/deep: [[1, 9], [2]] [[1, 9], [2]] [[1], [2, 8]]
container bytes tuple/list (1m): 8000040 8000056
create tuple/list seconds: 0.0006 0.0006 index seconds: 3.9e-06 6e-07
mutable item in tuple: ('stop', [1, 2, 3])
```

Q3: Dictionaries: Farmer Helpline Complaint Analysis

AIM:

Count frequent words, group complaints by district, remove stop words and compare dictionary, list-count and Counter approaches.

THEORY:

A dictionary lookup is O(1) average because a hash selects a bucket; collisions can affect worst cases. Repeated list.count for d distinct words costs O(n d), while one dictionary pass and Counter take O(n). Python 3.7+ preserves insertion order. keys(), values() and items() are live views. The top-k report sorts by descending count and alphabetical word for ties. A dictionary is the practical choice for 100,000 complaints per day; store counts and grouped rows in separate mappings. Punctuation is removed and text is lowercased before splitting.

PROGRAM CODE:

```
import collections, random, string, time
calls=['seeds not received seeds delayed','fertiliser price high','seeds price high subsidy not received']
STOP={'not','high'}
def frequencies(texts,stop):
 counts={}; groups={}
 for line in texts:
  district,crop,msg=line.split('|') if '|' in line else ('NA','NA',line)
  groups.setdefault(district,[]).append((crop,msg))
  for word in msg.lower().translate(str.maketrans('','',string.punctuation)).split():
   if word not in stop: counts[word]=counts.get(word,0)+1
 return counts,groups
def top(counts,k):return sorted(counts.items(),key=lambda x:(-x[1],x[0]))[:k]
print(top(frequencies(calls,STOP)[0],3))
words=('seed price delayed subsidy received crop help').split()
for n in (1000,10000,100000):
 data=[random.choice(words) for _ in range(n)]; distinct=set(data)
 t=time.perf_counter(); d={}
 for w in data:d[w]=d.get(w,0)+1
 td=time.perf_counter()-t
 t=time.perf_counter(); lc={w:data.count(w) for w in distinct}; tl=time.perf_counter()-t
 t=time.perf_counter(); c=collections.Counter(data); tc=time.perf_counter()-t
 print(n,'dict/list.count/Counter:',*(round(x,4) for x in (td,tl,tc)),'same:',d==lc==c)
print('empty/case/punct:',frequencies(['','SEEDS, seeds!'],set())[0])
```

OUTPUT:

```
[('seeds', 3), ('price', 2), ('received', 2)]
1000 dict/list.count/Counter: 0.0001 0.0 0.0001 same: True
10000 dict/list.count/Counter: 0.0005 0.0004 0.0002 same: True
100000 dict/list.count/Counter: 0.0052 0.0053 0.002 same: True
empty/case/punct: {'seeds': 2}
```

Q4: Sets: Elective Registration Clash Detection

AIM:

Find students in both, exactly one, or neither of two electives and measure set operations.

THEORY:

A set stores each hashable registration number once, so duplicate input is discarded. Use intersection (&), symmetric difference (^), and difference after union to produce the three reports. Venn regions are: overlap, both outside the overlap, and outside the union. Membership is O(1) average for a set versus O(n) for a list; nested list comparison is O(nm), sorting and merging is O(n log n + m log m). Sets cannot contain lists, but can contain tuples. Converting a list to a set removes duplicates and order.

PROGRAM CODE:

```
import time
ai={'001','004','007','009','004'}; iot={'002','004','009','010'}; all_students={f'{i:03}' for i in range(1,11)}
both=ai & iot; exactly_one=ai ^ iot; neither=all_students-(ai|iot)
print('clash:',sorted(both));print('exactly one:',sorted(exactly_one));print('neither:',sorted(neither))
print('method check:',both==ai.intersection(iot),exactly_one==ai.symmetric_difference(iot))
for n in (10000,100000,1000000):
 vals=list(range(n)); s=set(vals); target=n-1
 t=time.perf_counter(); _=target in vals; tl=time.perf_counter()-t
 t=time.perf_counter(); _=target in s; ts=time.perf_counter()-t
 print(n,'list/set membership:',round(tl,6),round(ts,6))
a=list(range(500));b=list(range(250,750))
t=time.perf_counter(); common=[x for x in a for y in b if x==y]; nested=time.perf_counter()-t
t=time.perf_counter(); fast=set(a)&set(b); hashed=time.perf_counter()-t
print('common small:',len(common),len(fast),'seconds:',round(nested,4),round(hashed,6))
print('empty/identical:',set()&iot,ai==ai.copy())
```

OUTPUT:

```
clash: ['004', '009']
exactly one: ['001', '002', '007', '010']
neither: ['003', '005', '006', '008']
method check: True True
10000 list/set membership: 3e-05 0.0
100000 list/set membership: 0.00037 0.0
1000000 list/set membership: 0.005143 1e-06
common small: 250 250 seconds: 0.003 5.9e-05
empty/identical: set() True
```

Q5: List Comprehension: Cleaning Sensor Readings

AIM:

Clean sensor errors, convert valid temperatures, identify alarm minutes, and compare three implementation styles.

THEORY:

A comprehension has the form [expression for item in iterable if condition]. It is concise for simple transformations; use a loop when logic has several steps. A dictionary comprehension can map minute to Celsius, and a set comprehension can collect distinct alarm values. A generator yields values lazily and uses O(1) container space; a list uses O(n). Each method is O(n) time. Measurements show that comprehension readability is useful but not always the best choice for complex logic; map/filter may be less clear.

PROGRAM CODE:

```
import random, time, sys
random.seed(8)
def loop(a):
 out=[]
 for c in a:
  if -50<=c<=50: out.append((c,c*9/5+32))
 return out
def comp(a):return [(c,c*9/5+32) for c in a if -50<=c<=50]
def mapped(a):return list(map(lambda c:(c,c*9/5+32),filter(lambda c:-50<=c<=50,a)))
readings=[random.uniform(-55,55) for _ in range(10000)]
print('identical:',loop(readings)==comp(readings)==mapped(readings),'alarms:',[i for i,c in enumerate(readings) if
  -50<=c<=50 and not 2<=c<=8][:5])
for n in (10000,100000,1000000):
 a=[random.uniform(-55,55) for _ in range(n)]; times=[]
 for fn in (loop,comp,mapped):
  t=time.perf_counter();r=fn(a);times.append(time.perf_counter()-t)
 print(n,'loop/comp/map-filter:',*(round(x,4) for x in times),'same:',loop(a)==comp(a)==mapped(a))
g=(c for c in readings if -50<=c<=50); print('container bytes
  list/generator:',sys.getsizeof(comp(readings)),sys.getsizeof(g))
print('all valid/invalid:',len(comp([3]*5)),len(comp([99]*5)))
```

OUTPUT:

```
identical: True alarms: [0, 2, 3, 4, 5]
10000 loop/comp/map-filter: 0.0009 0.001 0.0016 same: True
100000 loop/comp/map-filter: 0.0113 0.0124 0.0155 same: True
1000000 loop/comp/map-filter: 0.1468 0.1597 0.1882 same: True
container bytes list/generator: 75672 208
all valid/invalid: 5 0
```

Q6: Nested Data: A Railway Timetable

AIM:

Build a nested timetable, query and sort trains by departure, and compare indexed and unindexed station searches.

THEORY:

The outer dictionary supports train-number lookup; each train dictionary names fields; the ordered stop list preserves sequence; tuples keep each stop record fixed; a set answers operating-day membership. The third stop departure is timetable[train]["stops"][2][2]. A scan costs O(total stops) per query. A station index maps station to train numbers, reducing work to O(trains at station) after O(total stops) construction and extra O(total stops) memory. A flat tuple list is simple but requires a full scan each time.

PROGRAM CODE:

```
import time, sys
trains={}
for i in range(5):
 name=f'Train {100+i}'; stops=[('Chennai','08:00','08:05'),('Katpadi','10:00','10:05'),('Bengaluru','12:00','12:05')]
 trains[str(100+i)]={'name':name,'stops':stops,'days':{'Mon','Wed','Fri'}}
index={}
for number,train in trains.items():
 for station,arr,dep in train['stops']:index.setdefault(station,[]).append(number)
found=[(n,trains[n]) for n in index.get('Katpadi',[]) if 'Wed' in trains[n]['days']]
for n,t in sorted(found,key=lambda x:next(s[2] for s in x[1]['stops'] if
  s[0]=='Katpadi')):print(n,t['name'],t['stops'][1][2])
print('third stop departure:',trains['100']['stops'][2][2])
for n in (100,1000,10000):
 db={str(i):{'stops':stops,'days':{'Mon'}} for i in range(n)}
 ix={}
 for k,v in db.items():
  for s,a,d in v['stops']:ix.setdefault(s,[]).append(k)
 t=time.perf_counter();plain=[k for k,v in db.items() if any(s=='Katpadi' for s,a,d in
  v['stops'])];t1=time.perf_counter()-t
 t=time.perf_counter();indexed=ix.get('Katpadi',[]);t2=time.perf_counter()-t
 print(n,'plain/index seconds:',round(t1,5),round(t2,6),'index bytes:',sys.getsizeof(ix))
print('unknown station:',index.get('Nowhere',[]))
```

OUTPUT:

```
100 Train 100 10:05
101 Train 101 10:05
102 Train 102 10:05
103 Train 103 10:05
104 Train 104 10:05
third stop departure: 12:05
100 plain/index seconds: 3e-05 0.0 index bytes: 184
1000 plain/index seconds: 0.00057 1e-06 index bytes: 184
10000 plain/index seconds: 0.00306 4e-06 index bytes: 184
unknown station: []
```

Q7: Design a Contact-Merging Program from First Principles

AIM:

Define contact identity, normalize phone numbers, merge duplicate records and compare dictionary grouping with pairwise matching.

THEORY:

Input is a set of contact records; output is one record per normalized phone key with sets of names, numbers and emails. Pseudocode: for each contact, normalize phone; choose phone or email key; get or create a merged record; add nonempty fields to its sets. This gives O(n) average time and O(n) storage. Pairwise comparisons cost O(n^2); the 100,000-contact estimate is extrapolated from the 1,000-contact measurement because a direct run is impractical. Different people with one shared number can be merged incorrectly; this is the remaining identity limitation.

PROGRAM CODE:

```
import random, re, time
contacts=[{'name':'Dr. Ravi','phone':'+91 98400 12345','email':'ravi@x.in'}, {'name':'ravi
  kumar','phone':'9840012345','email':'rk@x.in'}, {'name':'Maya','phone':'9876500000','email':''}, {'name':'No
  phone','phone':'','email':'n@x.in'}]
def norm(phone):return ''.join(ch for ch in phone if ch.isdigit())[-10:]
def merge(rows):
 out={}
 for c in rows:
  key=norm(c.get('phone',''))
  if not key: key='email:'+c.get('email','').lower()
  if not key: continue
  r=out.setdefault(key,{'names':set(),'phones':set(),'emails':set()})
  if c.get('name'):r['names'].add(c['name'].strip().lower())
  if c.get('phone'):r['phones'].add(c['phone'])
  if c.get('email'):r['emails'].add(c['email'].lower())
 return out
merged=merge(contacts);print('merged:',merged);print('manual check:',len(merged)==3,norm('+91 98400 12345') in merged)
for n in (1000,100000):
 data=[{'name':f'Person{i% (n//2)}','phone':f'+91 98{i:08d}'} for i in range(n)]
 t=time.perf_counter();res=merge(data);td=time.perf_counter()-t
 print(n,'dictionary seconds:',round(td,4),'groups:',len(res))
 if n==1000:
  t=time.perf_counter();pairs=sum(1 for i in range(n) for j in range(i) if
  norm(data[i]['phone'])==norm(data[j]['phone']));tp=time.perf_counter()-t
  print('pairwise 1000 seconds:',round(tp,4),'pairs:',pairs,'100000 quadratic estimate
  seconds:',round(tp*(100000/1000)**2,1))
```

OUTPUT:

```
merged: {'9840012345': {'names': {'ravi kumar', 'dr. ravi'}, 'phones': {'9840012345', '+91 98400 12345'},
  'emails': {'ravi@x.in', 'rk@x.in'}}, '9876500000': {'names': {'maya'}, 'phones': {'9876500000'}, 'emails':
  set()}, 'email:n@x.in': {'names': {'no phone'}, 'phones': set(), 'emails': {'n@x.in'}}}
manual check: True True
1000 dictionary seconds: 0.0015 groups: 1000
pairwise 1000 seconds: 0.65 pairs: 0 100000 quadratic estimate seconds: 6499.7
100000 dictionary seconds: 0.3159 groups: 100000
```